Store Runner

Rakhat Bektas

Abstract - The game Store Runner is a Unity based game that uses procedural generation to make a different and interesting experience. The game represents a maze like store environment where the player a young child must go and retrieve an item for their mother. The disorientation and urgency of the towering shelves and random layouts add to the challenge. The procedural maze guarantees that no two playthroughs are the same, increasing replayability. Procedurally generated mazes, randomly placed target items, AI driven enemies to act as obstacles and basic player movement mechanics are key gameplay features. In this project, procedural techniques contribute to the possibility of simple games with dynamic experiences.

I. Introduction

Store Runner is a game created in Unity. It has a procedurally generated maze which imitates store environment. The task for the player, who plays as a young child, is to go get an item inside of a maze like store layout for the child's mother who is waiting at the counter. Urgency and a sense of scale are key to the game design, as towering store shelves makes the familiar environment into a disorienting maze from the child's point of view. The maze is procedurally generated, and the store shelves are used as the walls of the maze, ensuring that no two playthroughs are the same, adding to the replayability and for a new challenge every time. Randomized layout makes this a good candidate for procedural generation, with every playthrough being different from the previous one.

The main game elements include target item, adults as enemies, maze and player. The maze spawns the target item the child's mother needs somewhere randomly in the maze at least 40% away from starting position, and the player must find and retrieve it before returning to the spawn. There are adults, who act as obstacles, rather than active threats, spawn and move randomly throughout the maze, and if the player bumps into them, they lose one of their three hearts and game is over if they lose all three. The store shelves rise to create the towering maze, which is disorienting and sized according to difficulty level. The player character is short as a child and has limited number of hearts.

Right now the game's state is just a base, it has almost all the features that are intended but lacks textures, animation and UI.

II. Literature/Ludic Review

This game is about bringing back the feeling of being a child, when the store is a massive maze and adults are intimidating. To reinforce that point of view, all text in the game will be gibberish to represent a child's inability to read, and the towering shelves make the game's very familiar store environment feel disorienting. "Among the Sleep" is a game that captures the vulnerability and wonder of being small in a large, mysterious world, and this idea of seeing the world through a child's eyes is the inspiration. My game also uses scale, simplicity, and perspective to immerse players in the mindset of a child, much like "Among the Sleep".

It adds to the experience as the store layout is always procedurally generated, the environment is always unpredictable and unfamiliar, just as children experience new places. The adults in the game aren't supposed to be real threats, they're supposed to be obstacles, because kids often think adults are scary but don't really understand them.

III. Methodology

The game has four main features: procedural maze generation, enemies, target item, player movement.

By following the video on YouTube "MAKE A MAZE IN UNITY" by b3agz gave the base to the project. The video describes and gives step by step guide on how to create a maze in Unity. After that new assets were added from asset library, "Grocery Store Furniture Pack Vol. 1" by Cybernetic Walrus. The assets were used to replace the wall in the maze as shown in Figure 1.

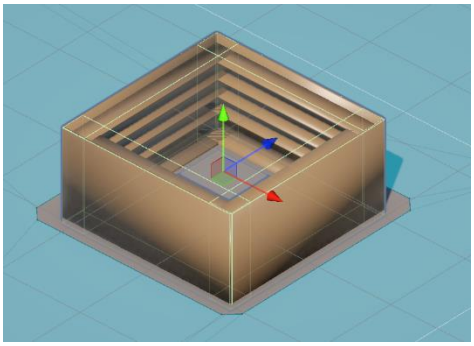


Figure 1. Cell asset

Also added the target item as a green cube that player would need to retrieve and the enemies(see Figure 2) which are just red cylinders(see Figure 3).

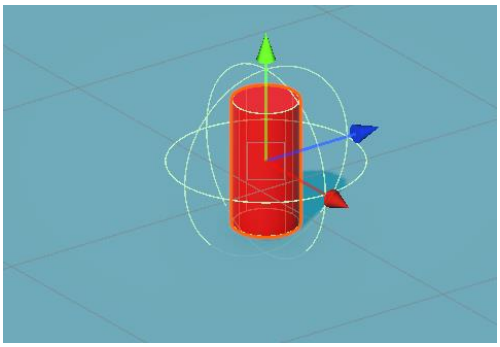


Figure 2. Enemy model

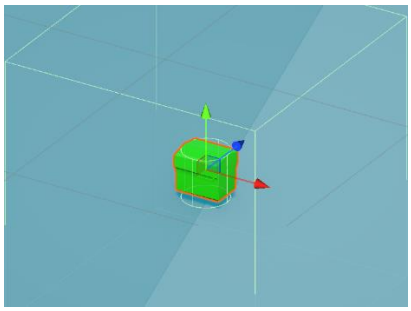


Figure 3. Target item

After doing the fixes and adding extra things such as enemies, entrance and target item to the generation, here is a breakdown of the main script:

The Maze generation involves creating a grid and then using an algorithm to carve out a path through it. The process begins with initializing the maze's dimensions and setting up a grid where each cell can either be a wall or a path. A common approach is to start from an initial cell and use a maze generation algorithm, such as Depth-First Search to explore neighboring cells. The algorithm randomly selects neighboring cells to move to, marking them as part of the maze path and removing walls between connected cells to create passages. This ensures that the maze is solvable and has a single connected path from the start to the end.

The script may also handle edge cases, such as ensuring there is a clear entrance and exit, and that all parts of the maze are reachable. Once the maze is fully generated, it is output in a readable format, which could be a visual representation in the console, a file, or a graphical interface. Optional features might include solving the maze, adding complexity, or allowing customization of the maze's size and difficulty. The maze can be customized and be different sizes and have different amount of enemies, like shown in Figure 4 you can see a maze 10x10.

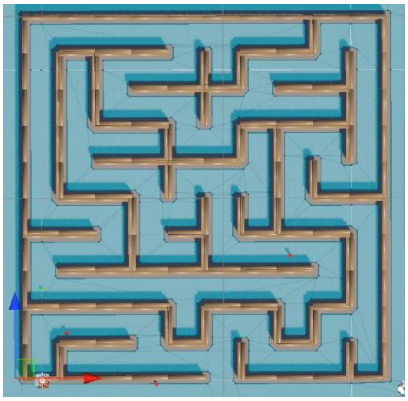


Figure 4. Generated maze 10x10

The enemy AI is simple but yet gets the job done in creating an adults that are doing their own thing. Enemies can walk and also damage the player when touched. In the future getting close to an enemy will turn into interaction with dialogs but it is fine like this for now. Here is explanation of enemy AI:

The EnemyMovement class implemented into Unity controls the movement of an enemy character with the help of the NavMeshAgent. The class begins with the maze’s size and the speed of the enemy. Similarly, during initialization, it sets a target position at a random place within the maze space. In the Update method, which is invoked on every frame, the code checks whether the enemy is almost at the current target position. If so, this invokes a method that generate a new target position within the range of the maze’s width and height and then set the NavMeshAgent to navigate towards this target. This leads to the enemy shifting to arbitrary positions in the maze time after time consecutively.

Target item appears in a random location in the maze but far enough to make it challenging to find. Player can interact with the object by moving close and picking it up. To implement this, a video tutorial

was followed on YouTube “How To Pick Up an Item - Unity” by Gunzz, in the video author explain the script and showed how to add text that will appear when you come close to the object. Here is an explanation of the script:

In Unity, the targetItem class manages player interaction with a collectible item. The OnTriggerStay method is repeatedly called when the player enters the item’s trigger area. If (by checking the tag) the player is detected, they will get a UI text of an option to pick up the item(see Figure 5). Then, if the player presses the 'E' key, the item is deactivated and a hidden object (which represents the item in the player's hands) is activated(see Figure 6). The player's hierarchy hides this hidden object. The UI text is hidden again if the player leaves the trigger area. Helper methods within the class are used to find the UI text and the hidden object within the player’s hierarchy.

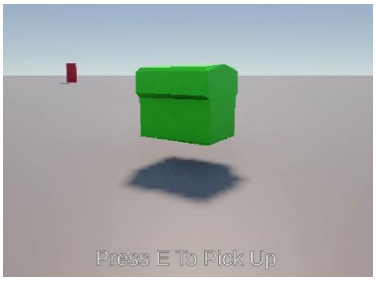


Figure 5. “Press E To Pick Up” UI text

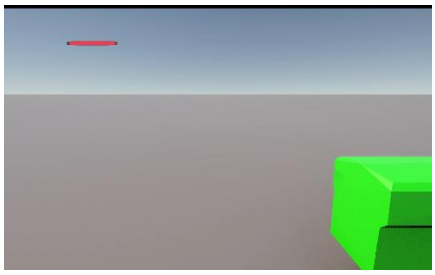


Figure 6. Item is in the hands

The last part is the player movement which is fairly simple. Player characters can move left, right, forward and back, also jump but the value of jump was lowered because it is not a necessary option for this environment. To implement this a video tutorial was followed on YouTube “UNITY 3D PLAYER MOVEMENT in 2 MINUTES! FPS Shooter” by Brogrammer, the author explained the script and showed how it works. Here is an explanation of the code:

In Unity the PlayerMovement class is used to control the player’s movement with the CharacterController component, as well as controlling the camera. The player can walk, run, jump, crouch and look around in the class. It first locks the cursor and sets the character default height and center. In the Update method, we calculate the movement direction of the player from input forward, backward and sideways. If the Left Shift key is held, the player can run(turned off); and if the Jump button is pressed while grounded, the player can jump. When the player is not grounded gravity is applied. The 'R' key is used to crouch, changing the character's height and speed. The calculated direction is used to move the player by the CharacterController. Also, the camera's Y axis is controlled by the mouse's Y input, and the player's X axis is controlled by the mouse's X input, so the player can turn around.

IV. Experimental Settings

For the evaluation of the game, the game test by volunteers were performed. A maze of 20 by 20 cells and 3 enemies was used in the testing. The volunteers were given a quick introduction of the game and controls information. All they needed to do is go inside the maze and retrieve the target item while avoiding the enemies. During the tests time was recorded and averaged about 4 to 6 minutes for this size of a maze.

Each playthrough differed from one another. There were 3 volunteers where one from two had completely different experince. First two testers had a fast time of completion and barely struggled with enemies where one even got hit by the enemy and have not noticed that.

However, the third tester had a the longest time to complete the maze. In the beginning the maze had three ways to go, the left one had the object and the right and center only had a dead ends. The tester chose to go to the right first and did not have any luck in fiding the target item and also have not seen any enemies. After returning back to the enterence he chose to go to the center but also no luck there, then he decided to go to the left way and found the target item and two enemies close to it. The tester then asked is it possible that the enemies will protect the item where answer to that question was no because the enemy AI does not have that in the code to guard the target item. After struggling with trying to go past the enemies the tester was observing the enemy movement and tried to find the window to slip through. The tester was able to retrieve the target item back to the center.

These different experiences with the game showed that the procedural generation works in creating a unique run each time.

V. Results and Discussion

Next step in development was to get volunteers' point of view on the game and ask a simple question "What do you think the game needs to change or add?" and the answers were collected. Here are the 5 answers by the testers:

- Lower the speed of the player character.
- Increase the height of the player character.
- Some kind of help to find the target item.
- Way to identify where you've been.
- Enemies should sometimes stay in place to imitate human behavior in the store environment.

The solution for the first request would be to decrease the speed of a player but also add a stamina bar and ability to sprint. This should make movement feel more diverse and add a layer to the player's thinking of when to drain the stamina.

The second request doesn't require any solution, because it is easy to adjust in the Unity. The way players see the maze and the store shelves is displayed in Figure 7. However, the third request is a bit controversial because it is supposed to be hard to find the target item otherwise why help player find it. For this request there are multiple solutions to consider. The first solution might be to add some kind of buff that player can find inside of the maze that will pin the location of the target item for a split second. This might help the player find the target item but also still be difficult to find it while only knowing the direction. The second solution to consider might be to add sections and make them a bit different so that when you see shelves full of bread the player would know that he/she should go in that direction because the target item is represented as a bread.

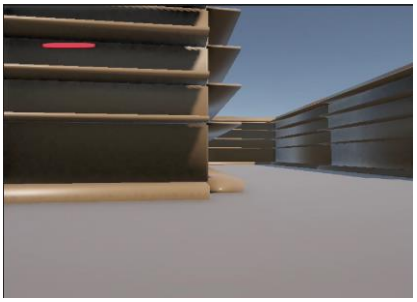


Figure 7. Player's point of view

The solution for the fourth request would be to add variety to the shelf to make them differ from one another, this could be achieved after implementing procedural shelf generation which is planned in the future. The last request should not be that hard to make, it will only need to add some lines of code to the enemy AI.

VI. Conclusion

The Store Runner, demonstrates how procedural generation can be used to create dynamic and replayable gameplay. The game combines procedurally generated maze, random placement of target item and simple AI driven obstacles to offer a new challenge every time. The game is currently in a base state with functional mechanics, and future improvements will include textures, animations, UI, and other gameplay features like stamina, enemy interaction, and more unique maze pieces. This project shows how procedural content generation can improve both technical and creative aspects of game design.

IV. References

[1] Krillbite Studio, "Among the Sleep", *Krillbite Studio*, May 29, 2014. Available: https://store.steampowered.com/app/250620/Among_the_Sleep__Enhanced_Edition/

[2] b3agz, "MAKE A MAZE IN UNITY", *YouTube*, Aug 30, 2022. [Online]. Available: https://www.youtube.com/watch?v=TMOEYdV4Ot4&pp=ygUKbWF6ZSB1bml0eQ%3D%3D

[3] Cybernetic Walrus, "Grocery Store Furniture Pack Vol. 1", *Unity Asset Store*, Dec 10, 2022 [Online]. Available: https://assetstore.unity.com/packages/3d/props/grocery-store-furniture-pack-vol-1-238689.

[4] Gunzz, "How To Pick Up an Item – Unity", *YouTube*, Oct 10, 2022. [Online]. Available: https://www.youtube.com/watch?v=zEfahR66Pa8

[5] Brogrammer, "UNITY 3D PLAYER MOVEMENT in 2 MINUTES! FPS Shooter", *YouTube*, May 11, 2023. [Online]. Available: https://www.youtube.com/watch?v=1uW-GbHrtQc